



GO FISH 2.0

Autonomous Fish-Following Robot

HSV vision + LiDAR sensing on a Raspberry Pi, driving a VEX V5 chassis through a maze while tracking a goldfish in real time.

Madison Carney & Alexis Phan

Ocean Lab, P6

Navya Arora & Esha Madamalla

Sys Lab, P7

Objective

Build an autonomous robot that navigates a maze while tracking a live goldfish target in a tank above it — using HSV color vision for the fish and LiDAR for the walls, all running in real time on a Raspberry Pi.

The same robot doubles as the data-collection rig for an oceanography study on goldfish learning.

Robotics

Computer Vision

Oceanography

Hypothesis



If goldfish are trained with spaced repetition and built-in breaks,
then they will learn the task faster and retain the association longer than fish trained in longer, less frequent sessions.

The robot serves as the consistent measurement instrument that makes this comparison possible — every fish is tracked the same way, every session.

Justification — Biological



- Goldfish navigate using landmarks, environmental cues, and associative memory.
- Their lateral pallium is homologous to the mammalian hippocampus — the same region responsible for spatial memory and navigation.
- Prior work has already shown that goldfish can be taught to navigate to a target zone (2022 navigation experiment).
- We extend this line of work by measuring movement objectively, instead of relying on visual observation.

Takeaway: there is biological reason to expect goldfish to learn a spatial task — and reason to measure it more precisely.

Justification — Technical



- Manual observation of fish movement is inconsistent, time-consuming, and imprecise.
- An HSV-based computer-vision system enables continuous, real-time tracking of the fish's position from an overhead camera.
- Adding a 360° LiDAR lets the robot map the maze around it without colliding into walls — something a camera alone cannot reliably do.
- Running everything on a Raspberry Pi keeps the system inexpensive, repeatable, and accessible.
- Together: a quantitative, automated way to collect movement data across many fish and many sessions.

Goal of the technical stack: reliable data, not impressive ML. The simplest detector that works is the right detector.

Relevant Studies



Crawford et al.

FINDING

Structured training schedules improve learning speed and retention in fish.

RELEVANCE

Justifies our two training schedules.

LIMITATION

Relied on manual observation — limited precision.

Blane et al.

FINDING

Goldfish learn behaviors through social cues; movement reflects learning.

RELEVANCE

Confirms tracking movement is a valid proxy for learning.

LIMITATION

No quantitative, real-time tracking.

Yang et al.

FINDING

Spaced repetition improves memory and long-term retention.

RELEVANCE

Informs our spacing comparison.

LIMITATION

Not paired with automated behavioral tracking.

PART 1

Robot & Code

The hardware, the sensors, the loop that ties them together.

Chassis Design — Model 1



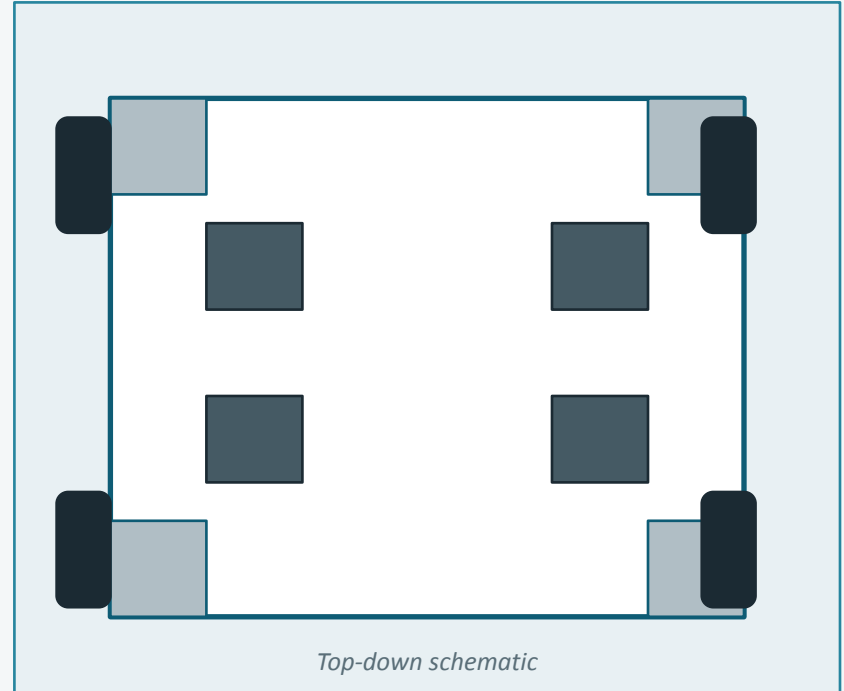
First Prototype

Pros

- Generous open frame — room for Pi, LiDAR, and cabling
- Four-wheel layout for stable forward motion

Cons

- Hard to secure the tank mount to the body
- Frame lacked enough support for the loaded water tank



Chassis Design — Model 2 (current)

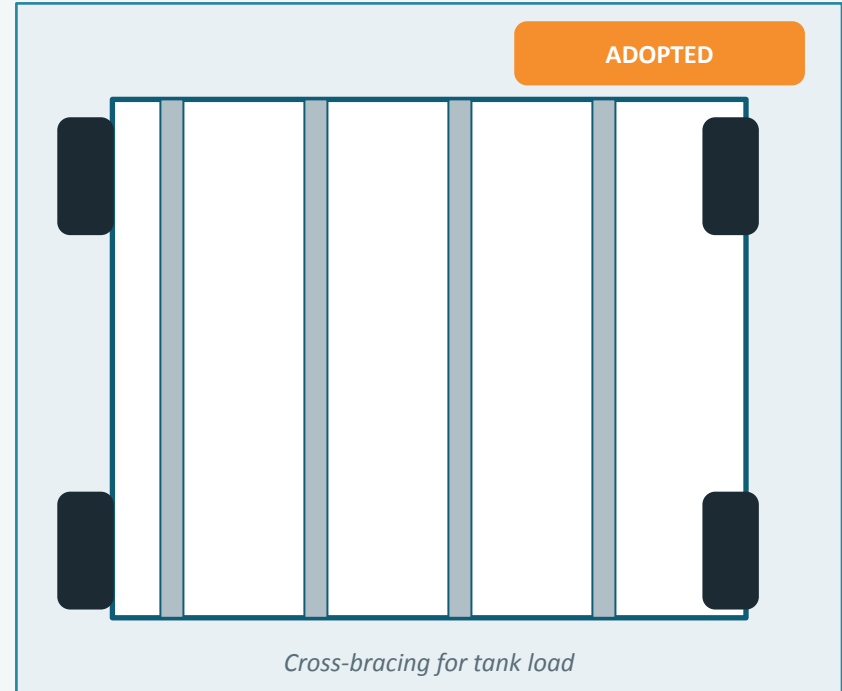
Reinforced Design

Pros

- Strong structural support — handles water-tank weight without flex
- More resistant to damage; the tank mount stays seated

Cons

- Denser frame leaves less room to route wiring
- Some Pi / LiDAR cabling had to be re-planned around supports



What Was Built Last Year



Last year's system was a starting point, not a finished product. Here is what actually ran on it:

- Detection used HSV color filtering — YOLO was tested but dropped because it was too slow on the Pi.
- Raspberry Pi processed the camera feed and sent commands to an Arduino Mega.
- The Arduino drove stepper motors using time-based pulses (run for X ms).
- LiDAR was proposed but never fully integrated into the loop.
- Movement was inconsistent — overshoot and drift were common, and there was no obstacle awareness.

Key takeaway: HSV ran reliably; the rest of the pipeline (motor control, obstacle handling) was the weak link.

How This Year Improves the System



Last Year vs. This Year

Three concrete changes — not a rewrite, a focused redesign

Last Year (2024–25)



Limitations

- YOLO tested but too slow on Pi → dropped during the year
- Stepper motors driven by time-based pulses (run for X ms)
- No real obstacle awareness — LiDAR proposed but not integrated
- Overshoot was common; corrections felt jerky
- Two control philosophies (vision + steppers) never fully unified

Key takeaway from last year's report

HSV color filtering ran reliably on the Pi.
YOLO was the bottleneck — not the motor controller.

This Year (2025–26)



Improvements

- HSV only — no YOLO, no second model, no ambiguity
- Arduino + steppers replaced by VEX V5 Brain + V5 motors
- LiDAR fully integrated — actual real-time obstacle avoidance

Single unified loop on the Pi

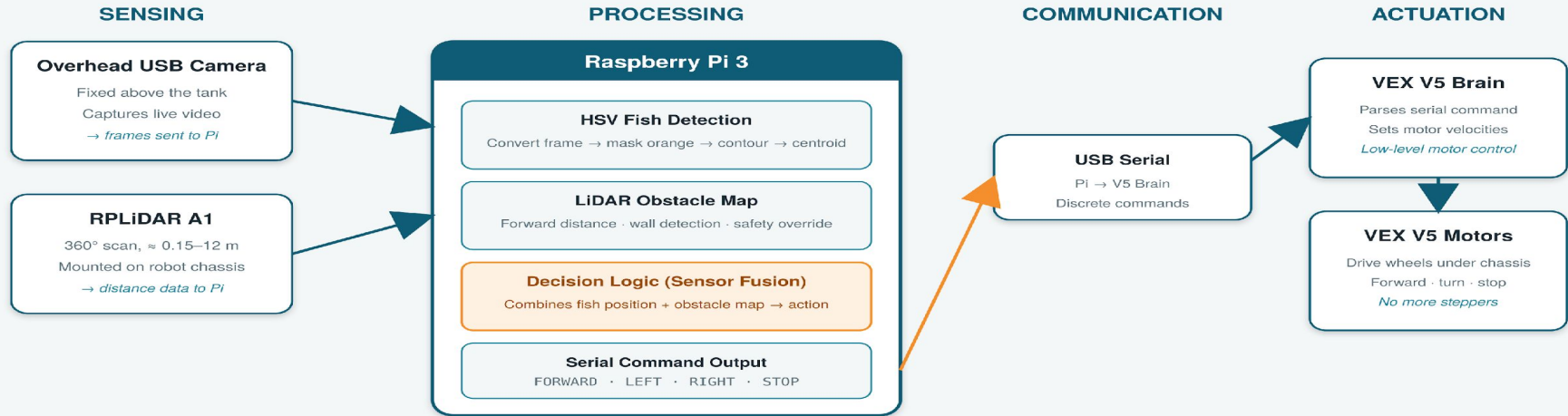
HSV finds fish · LiDAR enforces safety · Pi sends one of 4 commands
FORWARD · LEFT · RIGHT · STOP

System Architecture



Go Fish 2.0 — System Architecture

HSV vision + LiDAR sensing fused on Raspberry Pi → discrete commands → VEX V5



Single-detector pipeline — HSV runs every frame on the Pi

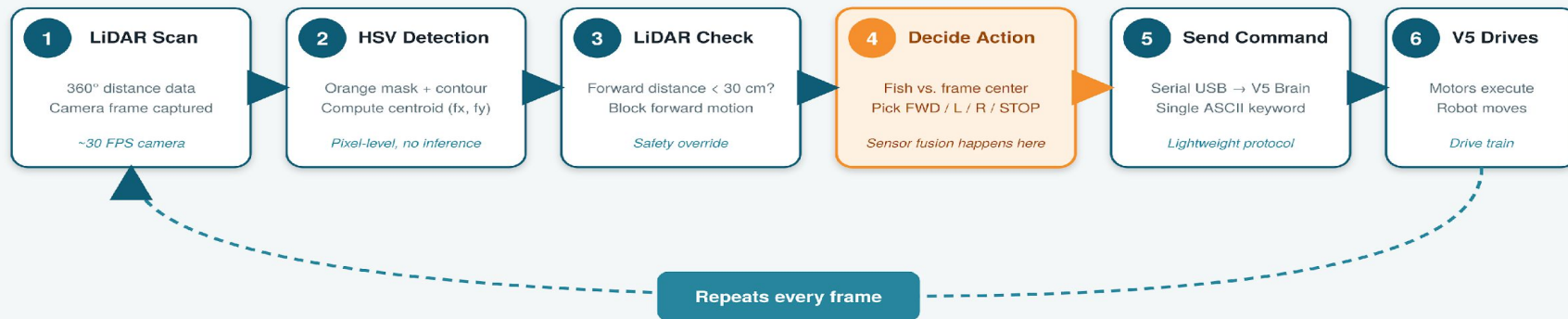
No neural network inference on-board. LiDAR runs in parallel as an independent safety/navigation channel.
The V5 Brain only receives discrete commands — Pi makes all sensing and planning decisions.

Real-Time Control Loop



Continuous Real-Time Control Loop

One single detection pipeline running on the Pi — no second model, no ambiguity



Why this pipeline is fast on a Raspberry Pi

HSV is pixel-level thresholding — no neural network inference. No model weights to load. No GPU required.
Detection cost is constant per frame; the loop runs in real time on Pi hardware that could not sustain YOLO.

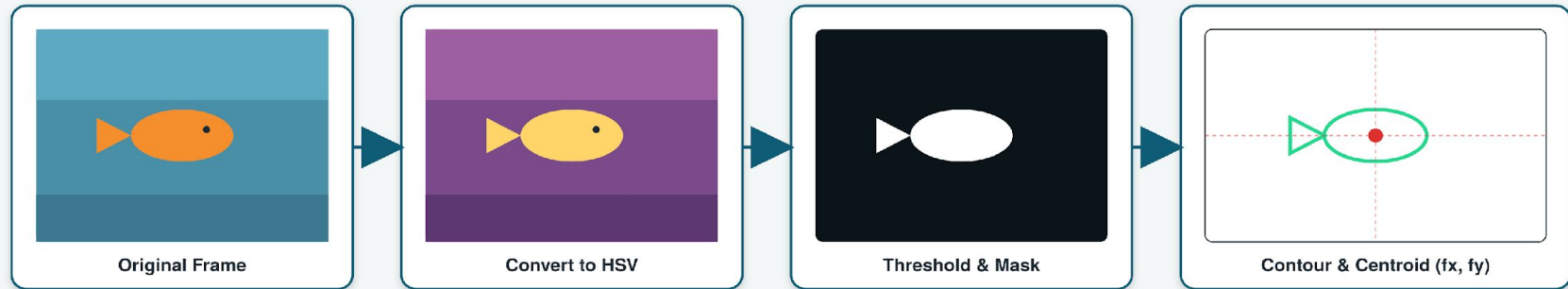
LiDAR safety check is independent of vision — it runs in parallel and only adds a few milliseconds.

HSV Fish Detection



HSV Fish Detection Pipeline

From camera frame to a single (fx, fy) point — runs in real time on Raspberry Pi



Why HSV separates orange goldfish cleanly

HSV separates color (Hue) from brightness (Value), so lighting changes affect V more than H — the orange hue stays stable.

Tuned range used on robot: $H \in [10, 35]$ $S \in [100, 255]$ $V \in [100, 255]$

Largest contour is taken; its centroid is the fish position handed to the decision logic.

Why HSV — and Only HSV



HSV gives us everything we need

- Pixel-level thresholding — no neural network inference
- No model weights to load — boots and runs immediately
- Very low CPU load — leaves headroom for LiDAR processing
- Real-time frame rate on Pi 3 — no GPU required
- Orange goldfish in a fixed tank = ideal target for color thresholding

Why we did not run YOLO this year

- YOLO inference on a Pi 3 is the bottleneck — ~0.2–0.5 s per frame (2–5 FPS).
- No control change downstream (velocity or discrete commands) can speed up detection — the slowest step sets the loop speed.
- Running HSV + YOLO together on every frame would be slower than YOLO alone — strictly worse than the choice we made.
- Our goldfish are a fixed bright color in a controlled tank, so HSV is sufficient on its own.

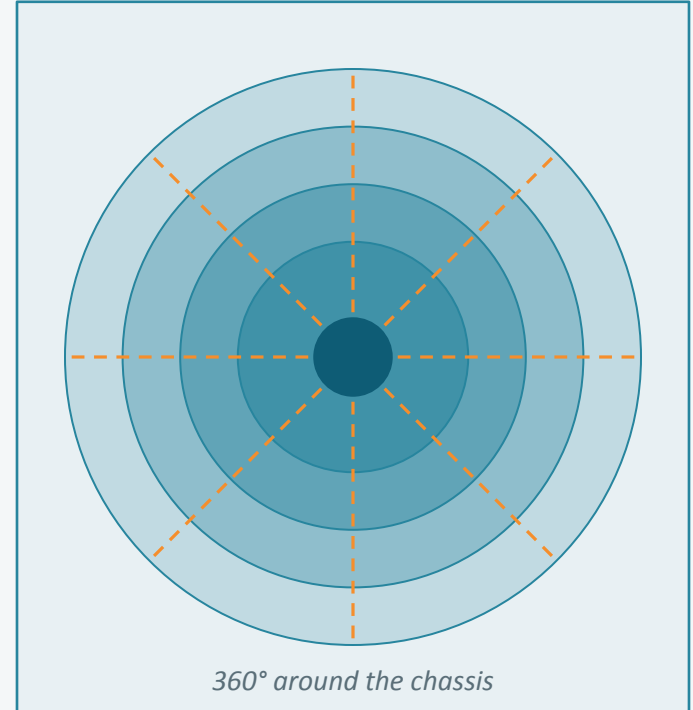
LiDAR — Hardware



RPLiDAR A1M8

WayPonDEV — 2D laser scanning sensor

Measurement	2D LiDAR (laser scanning)
Range	≈ 0.15 m – 12 m
Accuracy	± 1–2 cm short range; decreases with distance
Coverage	Full 360° scan, ~5–10 Hz
Output	Distance per angle, millimeters (we convert to cm)



LiDAR — Safety & Obstacle Avoidance



- Pi reads the forward-facing LiDAR distance every cycle.
- Distance is converted from millimeters to centimeters.
- If forward distance < 30 cm and the action is FORWARD, the Pi suppresses FORWARD and sends STOP instead.
- Below 20 cm, the Pi sends STOP unconditionally — emergency halt.
- The V5 Brain only ever sees post-safety commands; it does not need to know about LiDAR.

Pi-side safety override

```
fwd_cm = lidar_forward()
action = decide(fish_pos, frame)

if action == "FORWARD":
    if fwd_cm < 20:
        action = "STOP"
    elif fwd_cm < 30:
        action = "STOP"

send_serial(action) # → V5 Brain
```

Pi → VEX V5 Communication



Instead of computing motor velocities on the Pi, we let the V5 Brain handle low-level motor control. The Pi sends a single ASCII keyword over USB serial; the V5 program looks it up and drives the motors. This keeps the Pi free for vision and LiDAR.

FORWARD

Both drives forward at fixed speed

LEFT

Pivot in place — right > left

RIGHT

Pivot in place — left > right

STOP

All motors to zero

Why discrete commands here

- Easier to reason about while debugging the maze run
- No floating-point command stream to parse on the V5
- The V5's onboard PID handles the smoothness we used to fake in software

PART 2

Training

Six goldfish, two schedules, one tracking system.

The Goldfish



1

Mr. Hannum Jr



2

Melvin



3

Boo



4

Home Depot



5

Big Stuff



6

Bob

Materials & Methods



10 / 03 / 25

10-12 / 25

12 / 05 / 25

Wed & Fri



Tanks set up

2-month cycling

Goldfish introduced

Weekly training

Materials

- VEX V5 robotics kit (chassis + brain + motors)
- Raspberry Pi 3, USB camera, RPLiDAR A1
- 12" × 12" × 12" glass tank, filter, black foam board
- Water testing kit, goldfish pellets

Tank & Setup

- Tanks cycled for ~2 months before introducing fish
- Black foam board reduces reflections during tracking
- Overhead USB camera mounted above each session tank
- Same lighting setup for every session — keeps HSV thresholds stable

Training Groups



Group 1

Once weekly

Wednesdays only

10 minutes per fish

2 minutes on / 2 minutes off



3 goldfish in this group

Group 2

Twice weekly

Wednesdays & Fridays

5 minutes per fish

No breaks during the session



3 goldfish in this group

Training Schedule



Wednesday · Groups 1 + 2

10:25–10:40	Acclimate fish 1; set up robot & maze
10:40–10:55	Train fish 1; acclimate fish 4
10:55–11:10	Train fish 4; acclimate fish 5
11:10–11:25	Train fish 5; acclimate fish 2
11:25–11:30	Reset for Group 2
11:30–11:40	Train fish 2; acclimate fish 3
11:40–11:50	Train fish 3; acclimate fish 6
11:50–12:00	Train fish 6; cleanup, record data

Friday · Group 2 only

10:25–10:45	Acclimate fish 2; set up robot & maze
10:45–11:00	Train fish 2; acclimate fish 4
11:00–11:15	Train fish 4; acclimate fish 5
11:15–11:30	Train fish 5; cleanup, record data

Linking Training to Tracking



Training side

Fish learn to approach food cues at a specific zone of the tank.

- Spaced repetition + reward
- Predictable behavior over time
- Movement patterns reveal learning



Tracking side

Robot follows the fish in real time, recording where it goes.

- HSV detection every frame
- Position logged automatically
- Same instrument for every fish

Training ensures the behavior; tracking turns that behavior into data.

Results



91%

average HSV fish-tracking rate

Over all training sessions

across 6 fish, 2 schedules

What we observed

- The robot corrected its path in real time as the fish moved — no manual intervention during sessions.
- Repeated food-based conditioning increased movement toward the target zone over time, suggesting early associative learning.
- Shorter, more frequent sessions (Group 2) kept fish more responsive than longer sessions with breaks (Group 1).
- LiDAR successfully prevented the robot from contacting maze walls during sessions.

Conclusion



- Built an autonomous fish-following robot that runs entirely on a Raspberry Pi and VEX V5 stack — no Arduino, no steppers, no second detector.
- HSV color tracking proved sufficient for orange goldfish in a controlled tank; the simpler detector kept the loop real-time.
- Integrating LiDAR turned the robot from a follower into a maze navigator — the missing piece from last year is now part of every cycle.
- Used the robot as the measurement instrument for an oceanography learning study across two training schedules.
- Early evidence: shorter, more frequent sessions outperformed longer spaced ones with breaks.

The simplest detector that works is the right detector — pairing it with LiDAR did more for performance than any model upgrade would have.

Future Work

1

Predictive tracking

Add a Kalman filter so the robot can lead the fish slightly rather than chase it, smoothing motion when the fish darts.

2

Multi-fish tracking

Extend HSV detection to track multiple fish simultaneously, comparing individual learning curves directly.

3

Larger / dynamic mazes

Use LiDAR to build a live occupancy map for mazes too big to memorize, enabling reconfigurable test environments.

4

More training schedules

Run more spacing variants and longer studies to test how durable the learning effect is over weeks, not days.

5

Open-source release

Package the HSV + LiDAR pipeline and V5 communication code so other labs can run the same experiment.

Thank you.

We'd like to thank...

Dr. Stickler

for being our wonderful lab director

Dr. Yilmaz

for helping us with the code

Mr. Seyler

for help designing and building the robot

Mr. Hannum & Dr. Linch

for their aura

The Ocean Lab

for support and feedback throughout the year